

广东政策文本采集与分析系统

代钰堃

深圳大学微众银行金融科技学院

2026 年 6 月

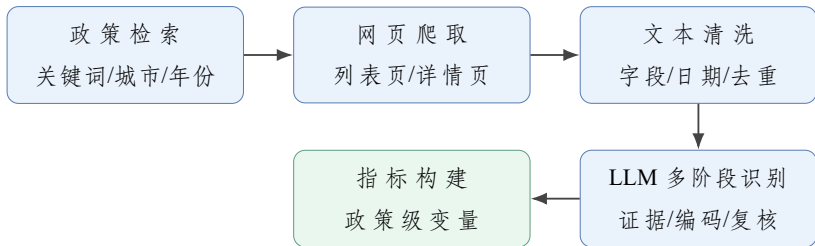


深圳大学
SHENZHEN UNIVERSITY



SWIFT
SHENZHEN UNIVERSITY WEBANK
INSTITUTE OF FINTECH
深大微众金融科技学院

系统整体框架



核心问题

本系统解决的是如何把分散在政府网站上的政策文本，系统化地转化为可复核、可统计、可进入回归分析的政策级指标。

数据采集模块

采集口径

项目	设置
数据来源	广东省政府政策文件检索接口
检索关键词	数字经济
时间范围	2017-2022 年
空间范围	广东 21 个地级市
保留字段	城市、标题、日期、年份、URL、正文
过滤规则	排除“已废止”等无效文本

表 1: 数据采集设置

代码入口

- `gd_digital_policy_crawler.py`
- 构造城市、年份、关键词组合；
- 调用政府检索接口获取列表；
- 进入详情页抽取正文；
- 汇总写入 Excel。

模块目标

本模块将分散在政府网站中的政策文本，转化为结构化、可追溯、可重复获取的政策样本库，为后续文本清洗与指标构建提供稳定的数据基础。

数据采集模块

核心控制参数

```
keywords = ["数字经济"]           # 检索关键词

start_year = 2017                  # 起始年份
end_year = 2022                    # 结束年份

excluded_title_markers = ["已废止"] # 预筛选排除词
columns = ["keyword", "city", "title",
           "date", "year", "url", "content"] # 输出字段
```

- 修改研究主题时，修改 keywords；
- 修改研究窗口时，修改 start_year 和 end_year；
- 修改无效文本规则时，扩展 excluded_title_markers；
- 输出字段固定，有利于后续清洗和 LLM 编码复用。

数据采集模块

适配复杂 TLS 和证书环境

- 政府网站常见证书链不完整、证书过期、TLS 握手异常；
- Python requests/OpenSSL 对部分站点兼容性较弱；
- 系统先走 requests，失败后用系统 curl 兜底；
- 证书异常时允许 curl --insecure 作为最后兜底。

关键代码逻辑

```
def get_detail_content(url):  
    try:  
        html =  
            fetch_detail_html_with_requests  
            (url)  
        return extract_detail_content(html)  
    except Exception as e:  
        if is_certificate_verify_error(e):  
            return try_curl_detail_content(  
                url, insecure=True) or ""  
        if is_tls_compat_error(e):  
            return (  
                try_curl_detail_content(url)  
                or  
                try_curl_detail_content(url,  
                    insecure=True) or ""  
            )  
        return try_curl_detail_content(url)  
        or ""
```

数据采集模块

预筛选逻辑

- 标题中必须出现当前关键词；
- 标题匹配前先去掉空白，兼容“数字经济”；
- 标题含“已废止”等标记时提前排除；
- 先过滤无效候选，再进入详情页抓正文。

城市识别算法

- 把标题和正文合并为待匹配文本；
- 遍历广东 21 个地级市名称；
- 命中“广州”或“广州市”等形式即归属该城市；
- 未命中时标记为“广东省级/未识别”。

```
def is_excluded_title(title):  
    return any(marker in title for marker in excluded_title_markers)  
  
def judge_city(title, content):  
    text = f"{title} {content}"  
    for city in cities:  
        if city in text or f"{city}市" in text:  
            return city  
    return "广东省级/未识别"
```

文本清洗与样本整理

原始文本的问题

- 网页正文可能不完整；
- Excel 表格中的正文可能只是网页片段；
- 标题和正文格式不统一；
- 日期格式可能不一致；
- 城市归属需要统一识别；
- 同一政策可能在多个页面重复出现。

清洗处理

- 标准化字段名和基础字段；
- 清洗空白字符和异常符号；
- 解析年份和发布日期；
- 生成稳定的 `policy_id`；
- 去重并标记疑似不完整文本；
- 调用 API 前按 URL 回溯抓取网页全文。

模块定位

清洗模块的目标不是改变政策含义，而是把网页文本整理成后续模型可以稳定读取的标准样本表。

文本清洗与样本整理：字段标准化和去重

样本整理入口

- 读取 Excel 并校验字段；
- 清理标题、正文、日期；
- 根据城市、标题、日期生成唯一 ID；
- 用去重规则保留一条主记录。

关键控制参数

```
DROP_DUPLICATE_POLICIES = True
# 是否按城市-标题-年份去重

required_columns = [
    "city", "title", "date",
    "url", "content" # LLM 编码必需字段
]
```

为什么生成 policy_id

policy_id 由 URL、标题等稳定字段生成，用于 JSONL 断点续跑、网页缓存更新、人工复核表回查和最终指标表合并。

文本清洗与网页补全

处理表格文本片段不全

- 表格中的 content 可能只是列表摘要或正文截断；
- 系统在调用 API 前读取 URL，重新抓取网页正文；
- 网页正文写入缓存，避免重复请求；
- 最终给 LLM 的文本记录来源：table 或 web。

网页补全控制参数

<code>FETCH_WEB_TEXT_BEFORE_API = True</code>	<code># API 调用前回溯网页全文</code>
<code>FORCE_REFETCH_WEB_TEXT = False</code>	<code># 是否强制重抓网页</code>
<code>WEB_FETCH_TIMEOUT = 15</code>	<code># 网页请求超时秒数</code>
<code>WEB_TEXT_MIN_GAIN_CHARS = 300</code>	<code># 绝对长度增益阈值</code>
<code>WEB_TEXT_MIN_GAIN_RATIO = 1.10</code>	<code># 相对长度增益阈值</code>
<code>SKIP_INCOMPLETE_TEXT_BEFORE_API = True</code>	<code># 跳过疑似残缺文本</code>

文本清洗与网页补全

决策机制

- 不盲目用网页文本替换表格文本；
- 只有网页文本明显更完整时才替换；
- 判断标准同时看“绝对增益”和“相对增益”；
- 替换后重新计算 `analysis\_text\_hash`，保证可追溯。

```
def should_use_web_text(table_text, web_text):  
    table_len = len(clean_text(table_text))  
    web_len = len(clean_text(web_text))  
    if web_len >= table_len + WEB_TEXT_MIN_GAIN_CHARS:  
        return True  
    return (  
        table_len > 0 and  
        web_len / table_len >= WEB_TEXT_MIN_GAIN_RATIO and  
        web_len >= 800  
    )  
  
final_text = web_text if should_use_web_text(table_text, web_text) else table_text
```

LLM 编码模块

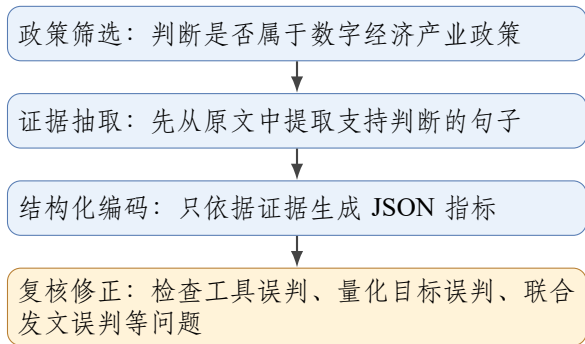
直接一次性编码的问题

- 模型可能跳过原文证据，直接给出看似合理的判断；
- 政策工具、量化目标、联合发文容易被过度推断；
- 输出字段多时，JSON 更容易缺字段或格式异常；
- 结果出错后，很难判断错误发生在哪一步。

设计思路

为了降低 LLM 幻觉，系统不让模型直接对全文一次性给出所有变量，而是采用“先筛选、再取证、再编码、再复核”的多阶段流程。

LLM 多阶段识别流程



这套流程把模型判断拆成可以检查的中间环节，后续发现问题时可以定位到筛选、证据、编码或复核中的具体步骤。

LLM 编码模块

Prompt 约束

- 明确数字经济产业政策口径；
- 要求输出 JSON；
- 给出字段示例和取值范围；
- 要求所有判断尽量回到原文证据。

程序约束

- 解析失败时进入容错解析；
- 每条结果立即写入 JSONL；
- 已有结果可断点续跑；
- API 失败保留错误信息。

API 与批处理控制参数

```

BASE_URL = os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com") # API 地址
MODEL = os.getenv("DEEPSEEK_MODEL", "deepseek-chat") # 模型名称

RUN_FULL_BATCH = False # 是否正式调用 API
ROW_LIMIT = None # 调试时限制条数
BATCH_SIZE = 50 # 中间快照批次大小
SLEEP_SECONDS = 0.5 # API 调用间隔
ENABLE_REVIEW_ROUND = True # 是否启用复核修正
    
```

指标体系

指标	含义
fiscal_tool	是否使用财政支持工具，如补贴、奖励、专项资金、税费优惠。
financial_tool	是否使用金融支持工具，如信贷、融资、基金、资本市场支持。
supply_tool	是否使用供给型工具，如平台、人才、技术、数据资源供给。
demand_tool	是否使用需求型工具，如试点示范、应用场景、政府采购。
regulation_tool	是否使用监管型工具，如标准规范、监管要求、数据安全。
target_specificity	是否包含明确的量化发展目标。
coordination_breadth	是否多部门联合发文。
tone_dummy	政策基调，主要区分支持性、中性和监管约束性表达。

指标定位

输出结果不是简单文本摘要，而是服务于论文描述统计、城市比较和后续回归分析的结构化变量。

指标体系

变量生成逻辑

- LLM 输出的 `tool_count` 先标准化为五类工具次数；
- 次数大于 0 时，生成对应的二元工具变量；
- `policy_tone` 映射为 `tone_dummy`；
- 量化目标和联合发文统一转成 0/1。

```
tool_df = pd.DataFrame(tool_counts.tolist()).reindex(
    columns=TOOL_CATEGORIES
).fillna(0).astype(int)

for category in TOOL_CATEGORIES:
    output_df[f"{category}_count"] = tool_df[category]
    output_df[f"{category}_tool"] = (tool_df[category] > 0).astype(int)

output_df["tone_dummy"] = output_df["policy_tone"].map(
    {"supportive": 1, "neutral": 0, "regulatory": -1}
)
output_df["target_specificity"] = output_df["target_specificity"].map(coerce_binary)
```

质量控制

模型输出控制

- JSON 格式约束，避免输出不可解析；
- 字段取值范围约束，减少自由发挥；
- 多阶段 Prompt，降低一次性判断风险；
- 可选复核阶段，修正常见误判。

程序运行控制

- API 失败重试；
- JSONL 断点续跑；
- 证据片段回查原文；
- 输出失败和排除原因，便于复核。

核心思路

系统不把 LLM 结果当作不可检查的黑箱，而是要求每个判断尽可能留下原文证据、状态标记和可追溯的中间结果。

质量控制

标题反向复核

- 识别“解读、图解、公示、名单、通知壳”等非政策文本；
- 避免把新闻、通知附件、名单公示误当成正式政策；
- 对疑似非政策文本降低进入主结果的概率。

筛选阈值

```
return bool(  
    is_policy == 1 and  
    positive_score >=  
        DIGITAL_POLICY_SCORE_MIN and  
    not_score <  
        TITLE_NOT_POLICY_SCORE_MAX and  
    positive_score - not_score >=  
        SCORE_MARGIN_MIN  
)
```

证据回查原文

```
def verify_evidence_against_text(policy_text, evidence):  
    text_norm = normalize_for_match(policy_text)  
    quotes = collect_evidence_quotes(evidence)  
    verified = sum(  
        normalize_for_match(q).replace("...", "") in text_norm  
        for q in quotes  
    )  
    return verified / len(quotes) if quotes else 1.0
```

系统输出

输出文件	主要内容
<code>gd21_policy_llm_results.jsonl</code>	完整 LLM 中间结果。包含通过、排除、跳过、错误信息、证据、模型输出和状态标记。
<code>gd21_policy_web_fulltext_cache.jsonl</code>	网页全文抓取缓存。避免重复请求，也方便回查每条 URL 的补全文本。
<code>gd21_policy_web_fulltext_review.csv</code>	网页抓取复核表。记录请求状态、正文长度、是否需要人工检查。
<code>gd21_policy_indicators_partial_preview.xlsx</code>	政策级指标预览表。包含基础字段、状态字段、证据字段和最终指标列。

口径说明

系统围绕四类文件落盘。被排除政策记录、API 失败记录和质量检查结果不拆成额外独立文件，而是集中保存在 LLM 结果 JSONL 和指标预览表的状态字段中。

总结

一条完整的数据生产链路

政策检索 → 网页爬取 → 文本清洗 → LLM 多阶段识别 → 指标构建

- 在采集端，系统保证政策样本来源清楚、可重复获取；
- 在清洗端，系统把网页文本整理成稳定的政策样本；
- 在识别端，系统通过证据抽取和复核降低 LLM 幻觉；
- 在输出端，系统生成可用于论文分析的政策级指标；
- 在质量控制端，系统保留中间状态、证据和错误信息，便于人工复核。

现有方法的局限与优化方向

现有方法的局限性

- 长文本处理压力：对于篇幅较长、结构复杂的政策文件，直接依赖 LLM 分析全文，可能带来成本、效率和稳定性问题；
- 候选信息定位仍可优化：当前主要依靠提示词范例引导 LLM 识别政策工具，但对长文本中高相关段落的前置定位还可以进一步加强；
- 隐含政策信息识别难度较高：部分政策措施并不会以典型表述出现，LLM 需要在较长上下文中判断其真实含义，容易增加识别难度。

下一步优化方向

- 引入预训练模型作为前置模块：借鉴相关文献，先用 BERT 等预训练模型从长文本中筛选高相关政策段落；
- 形成“预训练模型 + LLM”流程：预训练模型负责候选段落定位，LLM 负责复杂语义判断和结构化抽取；
- 提高处理效率与稳定性：减少无关文本输入，让 LLM 更集中地处理与政策工具相关的内容。

广东政策文本采集与分析系统

代钰堃

深圳大学微众银行金融科技学院

2026 年 6 月



深圳大学
SHENZHEN UNIVERSITY



SWIFT
SHENZHEN UNIVERSITY WEBANK
INSTITUTE OF FINTECH
深大微众金融科技学院